

Foundation of Blockchain Technology (IS501) **Pre-Reading Material – Lecture 20**

Errors in Solidity

Estimated Reading Time: 15–20 minutes

Handling Errors in Solidity

Introduction

Smart contracts may encounter unexpected situations such as invalid inputs, unauthorized users, insufficient balance, invalid addresses, or unexpected contract states. **Error handling** allows a Solidity contract to safely detect these situations and stop or reverse execution when necessary.

Proper error handling helps to:

- Maintain contract stability.
- Protect blockchain data integrity.
- Prevent invalid operations.
- Communicate the reason for failure.

Key Idea: *Plan for failure.*

Error Handling in Solidity

Earlier versions of Solidity used the throw statement to stop execution. However, throw is deprecated and should not be used in modern contracts.

Modern Solidity mainly provides three mechanisms:

Function

Main Purpose

require() Check conditions before execution

revert() Explicitly stop execution when a problem is detected

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



Function

Main Purpose

`assert()` Detect programming errors or impossible conditions

A simple way to remember them is:

require → **Prevent**

revert → **Stop and undo**

assert → **Detect internal errors**

The `require()` Function

`require()` is used to check a condition before executing important code. If the condition is false, the transaction is reverted.

Syntax

```
require(condition, "Error message");
```

Example

```
function withdraw(uint amount) public {  
    require(msg.sender == owner, "Not the contract owner");  
    require(amount <= balance, "Insufficient funds");  
  
    balance -= amount;  
}
```

Here, `require()` checks whether the caller is the owner and whether sufficient balance is available.

Common Uses

- Input validation
- Access control
- Ownership checks
- Balance verification
- Function preconditions



Analogy: Check a person's ID before allowing entry. If the ID is invalid, do not start the process.

The revert() Function

revert() is used to explicitly stop execution when a known problem is detected.

Syntax

```
revert("Reason for failure");
```

Example

```
function transfer(address recipient, uint amount) public {  
    if (msg.sender != owner) {  
        revert("Only owner can transfer");  
    }  
  
    // Continue transfer  
}
```

The if statement detects the problem and revert() stops the transaction.

revert() is particularly useful when conditions are checked inside a function or when different failure conditions need to be handled explicitly.

Key Point

When a transaction is reverted, the state changes made during that transaction are rolled back.

The assert() Function

assert() is used for conditions that **should always be true**. It is mainly used to detect programming errors, broken assumptions, or impossible states.

Syntax

```
assert(condition);
```

Example

```
function checkValue(uint a, uint b)  
    internal
```



```

pure
returns (uint)
{
    uint c = a + b;
    assert(c >= a);
    return c;
}
    
```

The programmer expects the condition to always be true.

Important Rule

Do **not** normally use `assert()` for user input validation.

Use:

`require(amount > 0, "Invalid amount");`

rather than:

`assert(amount > 0);`

Remember:

- `require()` → user/input condition
- `assert()` → developer/program logic check

Comparison of Error Handling Functions

Feature	<code>require()</code>	<code>revert()</code>	<code>assert()</code>
Main purpose	Validate conditions	Explicitly stop execution	Detect internal errors
Common use	Input/access control	Known failure	Invariants/programming bugs
Error message	Yes	Yes	No message parameter
State changes	Rolled back	Rolled back	Rolled back
Typical frequency	Common	Common	Rare



All three mechanisms can prevent an invalid transaction from completing, but they are intended for different situations.

State and Gas

When a transaction is reverted, state changes made during that transaction are undone. This helps maintain blockchain consistency.

However, a failed transaction can still consume gas because computational work was performed before the failure occurred.

Therefore:

- `require()` should be used to check important conditions early.
- `revert()` should be used when a known problem is detected.
- `assert()` should be used sparingly for internal conditions.

Important: A failed transaction does not mean that all gas used by the transaction is returned. Gas already consumed before the failure is generally still consumed.

Error Handling Strategy

A simple strategy is:

Step 1 – Prevent

Use `require()` for preconditions.

```
require(msg.sender == owner, "Not authorized");
```

Step 2 – Handle

Use `revert()` when a known problem is detected.

```
if (amount > balance) {  
    revert("Insufficient balance");  
}
```

Step 3 – Safeguard

Use `assert()` for conditions that should never be false.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



```
assert(totalSupply >= 0);
```

Remember

Prevent → Handle → Safeguard

require() → revert() → assert()

9. Deprecated throw

Older Solidity programs may contain:

```
throw;
```

throw was previously used to stop contract execution, but it is now deprecated.

Modern Solidity contracts should use:

- require()
- revert()
- assert()

according to the situation.

Example

```
function changeOwner(address newOwner) public {  
    require(  
        msg.sender == owner,  
        "Only current owner can change owner"  
    );  
  
    if (newOwner == address(0)) {  
        revert("Invalid address");  
    }  
  
    assert(owner != newOwner);  
  
    owner = newOwner;  
}
```



Explanation

- **require()** checks whether the caller is authorized.
- **revert()** handles an invalid new address.
- **assert()** checks an internal assumption.
- If an error causes the transaction to revert, the state changes made during that transaction are rolled back.

Best Practices

1. Use **require()** for input validation and access control.
2. Use **revert()** for explicit and known failure conditions.
3. Use **assert()** sparingly for internal consistency checks.
4. Provide meaningful error messages.
5. Test both successful and failed transactions.
6. Do not use deprecated **throw** in new contracts.
7. Check important conditions before performing expensive operations.

Key Takeaways

- Error handling is essential for reliable smart contracts.
- **require()** checks whether a condition is satisfied before proceeding.
- **revert()** explicitly stops execution and rolls back state changes.
- **assert()** detects internal programming errors and impossible conditions.
- **throw** is deprecated.
- Reverted transactions do not retain state changes made during the transaction.
- Failed transactions can still consume gas.
- The basic error-handling principle is:

Prevent → Handle → Safeguard

require() → revert() → assert()

